Buổi 10

Tính đa hình (Polymorphism)

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Liên kết sớm và liên kết muộn

1.1 Vấn đề: gọi hàm qua con trỏ lớp cơ sở
1.2 Liên kết sớm và liên kết muộn
1.3 Đa hình qua tham số hàm: tham chiếu và giá trị

2. Phương thức ảo

2.1 Khai báo phương thức ảo
2.2 Cơ chế bên trong: bảng phương thức ảo

3. Lớp trừu tượng

3.1 Phương thức thuần ảo và lớp trừu tượng
3.2 Ví dụ: danh sách hình Circle, Rectangle, Triangle
3.3 Ví dụ: phân cấp nhân viên với lớp trừu tượng

4. Các thành viên ảo khác

4.1 Toán tử ảo
4.2 Constructor ảo và destructor ảo
4.3 Nhận biết kiểu lúc chạy: dynamic_cast
4.4 Quản lý danh sách đối tượng đa hình

5. Tổng kết và thực hành

5.1 Tóm tắt buổi 10
5.2 Bài thực hành buổi 10 – Tính diện tích danh sách Shape bằng đa hình
5.3 Câu hỏi và bài tập

1.1 Vấn đề: gọi hàm qua con trỏ lớp cơ sở

- Buổi 9: con trỏ lớp cơ sở trỏ được tới đối tượng lớp dẫn xuất. Câu hỏi: gọi hàm **đã được định nghĩa lại** qua con trỏ đó thì **bản nào** chạy?

```
class A
{  public:
      void InTen() const { cout << "A"; }
};
class B : public A
{  public:
      void InTen() const { cout << "B"; }   // định nghĩa lại
};
int main()
{  B b;
   A *p = &b;        // con trỏ A trỏ tới đối tượng B
   A &r = b;
   p->InTen();       // ?
   r.InTen();        // ?
}
```

```
AA
```

- Dù p **thực sự** trỏ tới một đối tượng B, hàm của **A** vẫn được gọi.

- **Vì sao?** Trình biên dịch quyết định gọi hàm nào dựa vào **kiểu khai báo** của con trỏ (A*), ngay **lúc biên dịch**.
- **Hậu quả:** muốn xử lý danh sách các hình (tròn, chữ nhật, tam giác) bằng mảng `Shape *ds[]`, gọi `ds[i]->Area()` sẽ luôn chạy `Shape::Area()` ⇒ sai.
- **Cách làm "thủ công" kiểu C:** thêm trường `Loai` và `switch`:

```
switch (ds[i]->Loai)
{  case TRON:  …((Circle*)ds[i])->Area(); break;
   case CHUNHAT: …
}
```

  - Mỗi lần thêm loại hình mới phải sửa **mọi** switch ⇒ dễ sai, khó bảo trì.
- ⇒ Cần cơ chế để lời gọi hàm chọn đúng bản theo **kiểu thực sự** của đối tượng: **phương thức ảo**.

1.2 Liên kết sớm và liên kết muộn

Tiêu chí	Liên kết sớm (tĩnh – early binding)	Liên kết muộn (động – late binding)
Thời điểm xác định hàm được gọi	Lúc **biên dịch**	Lúc **chạy chương trình**
Căn cứ	**Kiểu khai báo** của con trỏ/tham chiếu	**Kiểu thực sự** của đối tượng được trỏ tới
Áp dụng cho	Hàm thường, hàm không ảo, đa năng hóa hàm/toán tử, template	Hàm **virtual** gọi qua **con trỏ** hoặc **tham chiếu**
Tốc độ	Nhanh nhất	Chậm hơn một chút (tra bảng hàm ảo)
Linh hoạt	Thấp	Cao – nền tảng của **đa hình**

Đa hình (thời gian chạy)

Khả năng **cùng một lời gọi** `p->Area()` thực hiện các hàm **khác nhau** tùy theo đối tượng mà p trỏ tới. C++ hiện thực bằng **phương thức ảo** + **con trỏ/tham chiếu lớp cơ sở**.

- **Điều kiện để có đa hình** – cần **đủ cả ba**:
  - Có **kế thừa** (public).
  - Hàm được khai báo **virtual** ở lớp cơ sở và được **định nghĩa lại** ở lớp dẫn xuất (cùng tên, cùng tham số, cùng const).
  - Gọi hàm qua **con trỏ** hoặc **tham chiếu** lớp cơ sở (gọi qua **đối tượng** thì vẫn liên kết sớm).

1.3 Đa hình qua tham số hàm: tham chiếu và giá trị

- Hàm nhận tham số kiểu **lớp cơ sở** dùng được cho **mọi** lớp dẫn xuất – nhưng chỉ đa hình khi truyền bằng **tham chiếu** hoặc **con trỏ**.

```
class Shape
{  public:
      virtual ~Shape() {}
      virtual double Area() const { return 0; }
      virtual const char *Name() const { return "Shape"; }
};
class Circle : public Shape
{  public:
      Circle(double r) : R(r) {}
      double Area() const override { return 3.14159 * R * R;
}
      const char *Name() const override { return "Circle"; }
   private:
      double R;
};
void InThamChieu(const Shape &s)
{  cout << s.Name() << " " << s.Area() << endl;  }
void InGiaTri(Shape s)              // truyền GIÁ TRỊ
{  cout << s.Name() << " " << s.Area() << endl;  }
Circle c(2);
InThamChieu(c);
InGiaTri(c);
```

```
Circle 12.57
Shape 0.00
```

- **InThamChieu:** s là bí danh của c – đối tượng **thực sự** là Circle ⇒ gọi Circle::Name, Circle::Area.
- **InGiaTri:** khi truyền giá trị, một đối tượng **Shape mới** được tạo bằng copy constructor từ c – phần Circle (R) bị **cắt** (slicing, buổi 9). Đối tượng này **là** Shape ⇒ không còn đa hình.
- **Quy tắc:** đối tượng đa hình luôn truyền bằng `const LopCoSo &` hoặc `LopCoSo *`; lưu trong mảng/danh sách dưới dạng **con trỏ** (`Shape *ds[]`), không phải mảng đối tượng `Shape ds[]`.
- Với Shape là lớp **trừu tượng** (mục 3), viết `void InGiaTri(Shape s)` sẽ **lỗi biên dịch** ngay – trình biên dịch giúp tránh lỗi cắt đối tượng.

2.1 Khai báo phương thức ảo

- Thêm từ khóa **virtual** trước khai báo hàm ở **lớp cơ sở**. Hàm cùng chữ ký ở lớp dẫn xuất **tự động** là ảo (có thể ghi lại `virtual` hoặc dùng `override` của C++11 cho rõ ràng).

```
class A
{  public:
      void InTen() const { cout << "A"; }
      virtual void InTenAo() const { cout << "A"; }
};
class B : public A
{  public:
      void InTen() const { cout << "B"; }
      void InTenAo() const override { cout << "B"; }
};
int main()
{  B b;  A *p = &b;  A &r = b;
   p->InTen();   r.InTen();       // liên kết sớm
   cout << " | ";
   p->InTenAo(); r.InTenAo();     // liên kết muộn
}
```

```
AA | BB
```

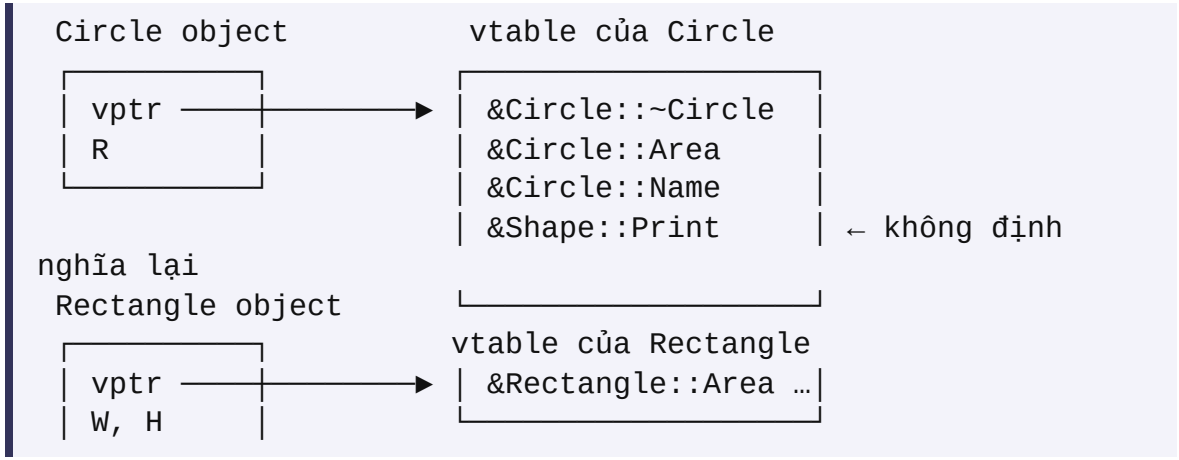
- **Quy tắc:**
  - Từ khóa virtual chỉ ghi trong **khai báo** trong lớp, không ghi ở định nghĩa ngoài lớp.
  - Hàm ở lớp dẫn xuất phải **trùng chữ ký** (tên, tham số, const). Sai một chi tiết (ví dụ quên const) ⇒ thành hàm **mới** che hàm ảo, đa hình **không** xảy ra.
  - `override` (C++11) yêu cầu trình biên dịch kiểm tra hàm thực sự ghi đè một hàm ảo – phát hiện ngay lỗi sai chữ ký.
  - Constructor **không** thể ảo; hàm static **không** thể ảo.

Ví dụ 10.1 – Lỗi sai chữ ký

```
class A { public: virtual void In() const; };
class B : public A { public: void In(); };  // thiếu const
A *p = new B;  p->In();    // gọi A::In – KHÔNG đa hình!
// Với override: void In() override;
// error: 'void B::In()' marked 'override', but does not
override
```

2.2 Cơ chế bên trong: bảng phương thức ảo

- Lớp có hàm ảo được trình biên dịch tạo một **bảng phương thức ảo (vtable)** – mảng địa chỉ các hàm ảo của lớp.
- Mỗi **đối tượng** của lớp có thêm một con trỏ ẩn **vptr** trỏ tới vtable của **lớp thực sự** của nó (được gán trong constructor).
- Lời gọi `p->Area()` được dịch thành: lấy `p->vptr` ⇒ tra ô Area trong vtable ⇒ gọi hàm tại địa chỉ đó.



Ví dụ 10.2 – Chi phí bộ nhớ

```
class NoV   { public: void f() {}          int x; };
class WithV { public: virtual void f() {}  int x; };
cout << sizeof(NoV) << " " << sizeof(WithV);
```

```
4 16
```

- WithV thêm con trỏ vptr (8 byte trên máy 64-bit) + căn lề ⇒ 16 byte.

- **Chi phí của hàm ảo:** mỗi đối tượng thêm một con trỏ; mỗi lời gọi thêm một lần tra bảng; không inline được khi gọi qua con trỏ. Thường **không đáng kể** so với lợi ích thiết kế.
- **Gọi hàm ảo trong constructor:** trong constructor lớp cơ sở, đối tượng **chưa** là lớp dẫn xuất (vptr đang trỏ vtable lớp cơ sở) ⇒ gọi bản của **lớp cơ sở**:

```
Base() { Ten(); }   ⇒ "Base ctor goi: Base"
Der()  { Ten(); }   ⇒ "Der ctor goi: Der"
```

Phương thức thuần ảo – lớp trừu tượng

- **Phương thức thuần ảo:** hàm ảo **không có thân** ở lớp cơ sở, khai báo bằng `= 0`: `virtual double Area() const = 0;`
- **Lớp trừu tượng:** lớp có **ít nhất một** phương thức thuần ảo. **Không thể tạo đối tượng** của lớp trừu tượng; chỉ dùng làm **lớp cơ sở** và để khai báo **con trỏ/tham chiếu**.
- Lớp dẫn xuất phải **định nghĩa tất cả** các phương thức thuần ảo, nếu không nó cũng là lớp trừu tượng. Lớp định nghĩa đủ gọi là **lớp cụ thể** (concrete class).

```
class Shape                        // lớp trừu tượng
{  public:
      virtual ~Shape() {}
      virtual double Area() const = 0;       // thuần ảo
      virtual const char *Name() const = 0;  // thuần ảo
      virtual void Print(ostream &os) const  // ảo, có thân
      {  os << Name() << ", area = " << Area();  }
};
Shape s;          // LỖI
Shape *p;         // hợp lệ: con trỏ
```

```
error: cannot declare variable 's' to be of
       abstract type 'Shape'
```

- **Ý nghĩa:** "hình học" nói chung không tính được diện tích – chỉ hình **cụ thể** mới tính được. Lớp trừu tượng định nghĩa **giao diện chung** (mọi hình đều có Area, Name, Print) mà **chưa** cài đặt.
- `Print` là ảo nhưng **có thân**, và gọi `Name()`, `Area()` – các hàm thuần ảo – sẽ chạy bản của lớp thực sự (mẫu thiết kế **Template Method**).

Loại hàm	Khai báo	Lớp dẫn xuất
Không ảo	`void F();`	Không nên định nghĩa lại
Ảo	`virtual void F();`	**Có thể** định nghĩa lại
Thuần ảo	`virtual void F() = 0;`	**Phải** định nghĩa lại

3.2 Ví dụ: danh sách hình Circle, Rectangle, Triangle

```
class Circle : public Shape
{  public:
      Circle(double r) : R(r) {}
      double Area() const override
      {  return 3.14159 * R * R;  }
      const char *Name() const override { return "Circle"; }
   private:
      double R;
};
class Rectangle : public Shape
{  public:
      Rectangle(double a, double b) : W(a), H(b) {}
      double Area() const override { return W * H; }
      const char *Name() const override { return "Rectangle";
}
   private:
      double W, H;
};
class Triangle : public Shape
{  public:
      Triangle(double a, double b, double c) : A(a), B(b),
C(c) {}
      double Area() const override          // Heron
      {  double p = (A + B + C) / 2;
         return sqrt(p * (p - A) * (p - B) * (p - C));  }
      const char *Name() const override { return "Triangle";
}
   private:
      double A, B, C;
};
```

```
int main()
{  Shape *ds[3] = { new Circle(2),
                    new Rectangle(3, 4),
                    new Triangle(3, 4, 5) };
   double tong = 0;
   for (int i = 0; i < 3; i++)
   {  cout << *ds[i] << endl;      // operator<< gọi Print ảo
      tong += ds[i]->Area();       // đa hình
   }
   cout << "Tong dien tich = " << tong << endl;
   for (int i = 0; i < 3; i++) delete ds[i];
}
```

```
Circle, area = 12.57
Rectangle, area = 12.00
Triangle, area = 6.00
Tong dien tich = 30.57
```

- Vòng lặp **không biết** và **không cần biết** mỗi phần tử là hình gì.
- **Thêm hình mới** (HinhThoi): chỉ viết lớp mới kế thừa Shape – vòng lặp, operator<< **không sửa một dòng** (nguyên lý **Mở – Đóng**: mở để mở rộng, đóng để sửa đổi).

3.3 Ví dụ: phân cấp nhân viên với lớp trừu tượng

```
class NhanVien                          // lớp trừu tượng
{  public:
      NhanVien(const char *t) { strcpy(Ten, t); }
      virtual ~NhanVien() {}
      virtual double TinhLuong() const = 0;
      virtual const char *Loai() const = 0;
      void In() const                   // không ảo, gọi hàm ảo
      {  cout << left << setw(8) << Ten << setw(10) << Loai()
              << right << setw(12) << TinhLuong() << endl;  }
   protected:
      char Ten[30];
};
class NVBienChe : public NhanVien
{  public:
      NVBienChe(const char *t, double hs) : NhanVien(t),
HeSo(hs) {}
      double TinhLuong() const override { return HeSo *
LuongCoBan; }
      const char *Loai() const override { return "BienChe"; }
      static double LuongCoBan;       // 2.340.000
   private:
      double HeSo;
};
class NVHopDong : public NhanVien    // lương ngày × số ngày
{  … double TinhLuong() const override
     {  return LuongNgay * SoNgay;  } … };
class NVThoiVu : public NhanVien     // quá 40 giờ: hệ số 1.5
{  … double TinhLuong() const override
     {  return SoGio <= 40 ? LuongGio * SoGio
              : LuongGio * (40 + (SoGio - 40) * 1.5);  } … };
```

```
NhanVien *ds[3] = { new NVBienChe("An", 2.34),
                    new NVHopDong("Binh", 300000, 22),
                    new NVThoiVu("Chi", 50000, 45) };
double tong = 0;
for (int i = 0; i < 3; i++)
{  ds[i]->In();
   tong += ds[i]->TinhLuong();
}
cout << "Tong quy luong: " << tong << endl;
for (int i = 0; i < 3; i++) delete ds[i];
```

```
An      BienChe        5475600
Binh    HopDong        6600000
Chi     ThoiVu         2375000
Tong quy luong: 14450600
```

- Chi làm 45 giờ: 50.000 × (40 + 5 × 1.5) = **2.375.000**.
- Hàm `In()` **không ảo** nhưng bên trong gọi `Loai()`, `TinhLuong()` **ảo** ⇒ vẫn in đúng cho mọi loại nhân viên.
- Thêm loại "Cộng tác viên" (lương theo sản phẩm): chỉ viết **một lớp mới**; vòng lặp tính quỹ lương **không đổi**.

- Toán tử `<<` **không thể** là hàm ảo (là hàm friend, không phải thành viên). Kỹ thuật: cài `operator<<` **một lần** cho lớp cơ sở, bên trong gọi một hàm thành viên **ảo** `Print` ⇒ `<<` hoạt động đa hình.

```
class Shape
{  friend ostream &operator<<(ostream &os, const Shape &s)
   {  s.Print(os);          // gọi hàm ẢO
      return os;
   }
   public:
      virtual void Print(ostream &os) const
      {  os << Name() << ", area = " << Area();  }
      ...
};
class Circle : public Shape
{  public:
      void Print(ostream &os) const override   // tùy chỉnh
      {  Shape::Print(os);
         os << ", radius = " << R;
      }
      ...
};
Shape &s = *new Circle(2);
cout << s;   // Circle, area = 12.57, radius = 2.00
```

- **Toán tử thành viên có thể khai báo ảo:** ví dụ `virtual bool operator==(const Shape &) const;` – nhưng tham số vẫn là kiểu lớp cơ sở ⇒ cần thận trọng khi so sánh hai đối tượng khác lớp.
- **Kỹ thuật "constructor ảo" (Clone):** constructor không ảo được, nhưng có thể tạo **bản sao đúng kiểu** qua con trỏ cơ sở:

```
class Shape
{  public:
      virtual Shape *Clone() const = 0;
};
class Circle : public Shape
{  public:
      Circle *Clone() const override
      {  return new Circle(*this);  }   // copy constructor
};
Shape *b = a->Clone();   // a trỏ Circle ⇒ b là Circle mới
```

- Dùng khi sao chép **danh sách đối tượng đa hình** (sao chép sâu mảng `Shape *`).

4.2 Constructor ảo và destructor ảo

- **Constructor không thể ảo:** khi tạo đối tượng phải biết **chính xác** lớp (`new Circle(2)`) – chưa có vptr để tra bảng.
- **Destructor nên ảo** khi lớp được dùng làm lớp cơ sở đa hình. Nếu không: `delete p;` với p kiểu cơ sở chỉ gọi destructor **lớp cơ sở** ⇒ phần dẫn xuất **không được dọn dẹp**.

Ví dụ 10.3 – Destructor không ảo

```
class Base1
{  public: ~Base1() { cout << "  ~Base1\n"; }  };
class Der1 : public Base1
{  public:
      Der1() { p = new int[100]; }
      ~Der1() { delete[] p; cout << "  ~Der1\n"; }
      int *p;
};
Base1 *q = new Der1;
delete q;
```

```
  ~Base1
```

- **~Der1 không chạy** ⇒ 400 byte của p bị rò rỉ; AddressSanitizer báo **new-delete-type-mismatch**.

Ví dụ 10.4 – Destructor ảo (lớp Shape)

```
class Shape { public: virtual ~Shape() { cout << "
~Shape\n"; } … };
class Circle : public Shape { public: ~Circle() { cout << "
~Circle\n"; } … };
for (int i = 0; i < 3; i++) delete ds[i];
```

```
  ~Circle
  ~Shape
  ~Rectangle
  ~Shape
  ~Triangle
  ~Shape
```

- Destructor ảo ⇒ `delete ds[i]` gọi đúng destructor của **lớp thực sự**, rồi tự động gọi destructor lớp cơ sở.

Quy tắc

Lớp có **ít nhất một hàm ảo** ⇒ hãy khai báo **destructor ảo** (`virtual ~Shape() {}`), kể cả khi thân rỗng.

4.3 Nhận biết kiểu lúc chạy: dynamic_cast

- Đôi khi cần biết con trỏ cơ sở thực sự trỏ tới lớp nào để gọi hàm **riêng** của lớp đó.
- `dynamic_cast<Circle *>(p)`: kiểm tra **lúc chạy**; trả về con trỏ hợp lệ nếu `*p` thực sự là Circle (hoặc lớp con của Circle), ngược lại trả về **NULL**. Chỉ dùng với lớp có hàm ảo.

```
for (int i = 0; i < 3; i++)
{  Circle *c = dynamic_cast<Circle *>(ds[i]);
   cout << ds[i]->Name() << ": "
        << (c ? "la Circle" : "khong phai Circle") << endl;
}
```

```
Circle: la Circle
Rectangle: khong phai Circle
Triangle: khong phai Circle
```

Cách ép	Kiểm tra lúc chạy	Khi ép sai
`(Circle *)p` (kiểu C)	Không	Hành vi không xác định
`static_cast<Circle *>(p)`	Không	Hành vi không xác định
`dynamic_cast<Circle *>(p)`	**Có**	Trả về NULL (với con trỏ)

- **Lời khuyên:** dùng dynamic_cast **hạn chế**; nếu phải kiểm tra kiểu ở nhiều nơi ⇒ dấu hiệu nên thêm một **hàm ảo** vào lớp cơ sở thay vì phân nhánh theo kiểu.
- `typeid(*p).name()` (thư viện `<typeinfo>`) cho tên kiểu thực sự – dùng để gỡ lỗi.

4.4 Quản lý danh sách đối tượng đa hình

- **Lớp quản lý danh sách** gói mảng con trỏ, chịu trách nhiệm tạo, sao chép (Clone), hủy các đối tượng:

```
class DanhSachHinh
{  public:
      DanhSachHinh() { N = 0; }
      ~DanhSachHinh()
      {  for (int i = 0; i < N; i++) delete ds[i];  }
      DanhSachHinh(const DanhSachHinh &d)     // sao chép sâu
      {  N = d.N;
         for (int i = 0; i < N; i++)
            ds[i] = d.ds[i]->Clone();         // đúng kiểu!
      }
      void Them(Shape *s) { if (N < 100) ds[N++] = s; }
      double TongDienTich() const
      {  double t = 0;
         for (int i = 0; i < N; i++) t += ds[i]->Area();
         return t;
      }
   private:
      Shape *ds[100];
      int N;
};
```

- **Clone:** `Shape *a = new Circle(1); Shape *b = a->Clone();` ⇒ b là một **Circle mới** (diện tích 3.14) dù chỉ biết a là `Shape *`.
- **Vì sao không sao chép `*ds[i]` trực tiếp?** `new Shape(*ds[i])` chỉ tạo được Shape (lại cắt đối tượng), và không làm được nếu Shape trừu tượng.
- **Tổng hợp các nguyên tắc đã học:**

Vấn đề	Giải pháp
Chọn đúng hàm theo kiểu thực sự	Hàm **virtual** + con trỏ/tham chiếu
Giải phóng đúng đối tượng dẫn xuất	**Destructor ảo**
Sao chép đối tượng đa hình	Hàm ảo **Clone()**
In đối tượng đa hình bằng `<<`	`<<` gọi hàm ảo **Print**
Danh sách nhiều loại	Mảng **con trỏ** lớp cơ sở + Quy tắc ba cho lớp quản lý

- C++ hiện đại: `vector<unique_ptr<Shape>>` tự giải phóng – nhưng nguyên lý giống hệt.

5.1 Tóm tắt buổi 10

Khái niệm	Ghi nhớ
Liên kết sớm	Chọn hàm theo kiểu khai báo, lúc biên dịch
Liên kết muộn	Chọn hàm theo kiểu thực sự, lúc chạy – cần **virtual** + con trỏ/tham chiếu
Phương thức ảo	`virtual` ở lớp cơ sở; lớp dẫn xuất trùng chữ ký; dùng `override`
vtable/vptr	Cơ chế cài đặt; mỗi đối tượng thêm một con trỏ ẩn
Thuần ảo, lớp trừu tượng	`= 0`; không tạo đối tượng; định nghĩa giao diện chung
Toán tử ảo	`operator<<` gọi hàm ảo `Print`
Constructor ảo	Không có; thay bằng `Clone()` ảo
Destructor ảo	**Bắt buộc** cho lớp cơ sở đa hình
dynamic_cast	Ép xuống có kiểm tra; NULL nếu sai kiểu

Câu hỏi nhanh

- Gọi `b.InTenAo()` trực tiếp qua **đối tượng** b (không qua con trỏ) là liên kết gì? → **Sớm** (trình biên dịch đã biết kiểu của b).
- Lớp trừu tượng có constructor không? → **Có** – được gọi khi tạo đối tượng lớp dẫn xuất để khởi tạo phần dữ liệu chung.

5.2 Bài thực hành buổi 10 – Tính diện tích danh sách Shape bằng đa hình

Yêu cầu

- 1) Lớp trừu tượng **Shape**: destructor ảo, các hàm thuần ảo `Area()`, `Perimeter()`, `Name()`; hàm ảo `Print(ostream &)`; `operator<<` gọi Print.
- 2) Các lớp **Circle**, **Rectangle**, **Triangle** (kiểm tra 3 cạnh hợp lệ); dùng `override` cho mọi hàm ghi đè.
- 3) Chương trình: nhập n hình (chọn loại, nhập kích thước), lưu trong mảng `Shape *` cấp phát động; in danh sách, tổng diện tích, hình có chu vi lớn nhất; sắp xếp theo diện tích giảm dần.
- 4) Đếm số hình tròn bằng `dynamic_cast`; sau đó thay bằng hàm ảo `bool LaHinhTron() const` và so sánh hai cách.
- 5) Thêm lớp **Square** kế thừa Rectangle và lớp **Ellipse** – kiểm chứng phần xử lý danh sách **không phải sửa**.
- 6) Thí nghiệm: bỏ `virtual` ở destructor, chạy với `-fsanitize=address` và ghi lại lỗi; bỏ `virtual` ở Area và giải thích kết quả mới.

Gợi ý kiểm tra

Circle(r = 2), Rectangle(3 × 4), Triangle(3, 4, 5): diện tích lần lượt **12.57**, **12.00**, **6.00**, tổng **30.57**; chu vi **12.57**, **14**, **12**.

5.3 Câu hỏi và bài tập

Câu hỏi

- 1) Phân biệt liên kết sớm và liên kết muộn. Điều kiện để có liên kết muộn?
- 2) Lớp trừu tượng là gì? Vì sao không tạo được đối tượng của lớp trừu tượng?
- 3) Vì sao constructor không thể ảo còn destructor nên ảo?
- 4) Làm thế nào để toán tử `<<` hoạt động đa hình?

Bài tập dự đoán kết quả (có đáp số)

```
class X { public: virtual void F() { cout << "XF "; }
                  void G() { cout << "XG "; F(); } };
class Y : public X { public: void F() { cout << "YF "; }
                     void G() { cout << "YG "; } };
int main()
{  Y y;  X *p = &y;  X x = y;
   p->F();  p->G();  x.F();  y.G();
}
```

→ **Đáp số:** `YF XG YF XF YG` – `p->G()` không ảo ⇒ X::G, nhưng bên trong gọi F() ảo ⇒ Y::F; `X x = y` bị cắt ⇒ `x.F()` là X::F.

Bài tập lập trình

- 1) Phân cấp **NhanVien** trừu tượng (`TinhLuong()` thuần ảo) với NVBienChe, NVHopDong, NVThoiVu; quản lý danh sách nhân viên bằng mảng con trỏ, tính tổng quỹ lương.
- 2) Phân cấp **DongVat** với hàm ảo `Keu()`, `DiChuyen()`; mô phỏng một vườn thú.